

SANGOLA MAHAVIDYALAYA, SANGOLA

Department of Computer Science and Application | B.C.A / B.Sc(CS) / B.Sc(ECS) - III

Subject: Dot Net Core | Practical Assignment - 02

Q1. Class `Book` with default constructor.

```
using System;
class Book {
    string title, author;
    // Default constructor
    public Book() {
        title = "Unknown Title";
        author = "Unknown Author";
    }
    public Book(string t, string a) { title = t; author = a; }
    public void Display() => Console.WriteLine($"Title: {title}, Author: {author}");
}
class Q1 {
    static void Main() {
        Book b1 = new Book(); // default
        Book b2 = new Book("C# in Depth", "Jon"); // parameterized
        b1.Display();
        b2.Display();
    }
}
// Output:
// Title: Unknown Title, Author: Unknown Author
// Title: C# in Depth, Author: Jon
```

Q2. Class `Student` with copy constructor.

```
using System;
class Student {
    public int RollNo;
    public string Name;
    public Student(int r, string n) { RollNo = r; Name = n; }
    // Copy constructor
    public Student(Student s) { RollNo = s.RollNo; Name = s.Name; }
    public void Display() => Console.WriteLine($"Roll: {RollNo}, Name: {Name}");
}
class Q2 {
    static void Main() {
        Student s1 = new Student(101, "Alice");
        Student s2 = new Student(s1); // copy
        s1.Display();
        s2.Display();
        s2.Name = "Bob"; // change copy - original unaffected
        Console.WriteLine("After change:");
        s1.Display(); s2.Display();
    }
}
```

Q3. Class `Car` with parameterized constructor.

```
using System;
```

```

class Car {
    string brand, model;
    double price;
    public Car(string brand, string model, double price) {
        this.brand = brand;
        this.model = model;
        this.price = price;
    }
    public void Display() =>
        Console.WriteLine($"Brand: {brand}, Model: {model}, Price: Rs.{price}");
    }
class Q3 {
    static void Main() {
        Car c1 = new Car("Toyota", "Innova", 1800000);
        Car c2 = new Car("Maruti", "Swift", 650000);
        c1.Display();
        c2.Display();
    }
}

```

Q4. Class `Employee` with constructor overloading.

```

using System;
class Employee {
    string name;
    double salary;
    public Employee(string name) { // only name
        this.name = name;
        this.salary = 0;
    }
    public Employee(string name, double salary) { // name + salary
        this.name = name;
        this.salary = salary;
    }
    public void Display() =>
        Console.WriteLine($"Name: {name}, Salary: Rs.{salary}");
    }
class Q4 {
    static void Main() {
        Employee e1 = new Employee("Ravi");
        Employee e2 = new Employee("Meera", 45000);
        e1.Display();
        e2.Display();
    }
}
// Output:
// Name: Ravi, Salary: Rs.0
// Name: Meera, Salary: Rs.45000

```

Q5. Class `Bank` with static constructor.

```

using System;
class Bank {
    public static string BankName;
    public static string BranchCode;
}

```

```

string customerName;
// Static constructor - runs once before any object is created
static Bank() {
    BankName = "State Bank of India";
    BranchCode = "SBI-001";
    Console.WriteLine("Static Constructor called: Bank initialized.");
}
public Bank(string name) { customerName = name; }
public void Display() =>
    Console.WriteLine($"Bank: {BankName} | Branch: {BranchCode} | Customer: {customerName}");
}
class Q5 {
    static void Main() {
        Bank b1 = new Bank("Amit");
        Bank b2 = new Bank("Priya");
        b1.Display();
        b2.Display();
    }
}

```

Q6. Method `BookTicket` with named parameters.

```

using System;
class Ticketing {
    static void BookTicket(string passengerName, string destination, string seatType = "General") {
        Console.WriteLine($"Passenger : {passengerName}");
        Console.WriteLine($"Destination: {destination}");
        Console.WriteLine($"Seat Type : {seatType}");
        Console.WriteLine("Ticket Booked Successfully!");
    }
    static void Main() {
        // Using named parameters - order doesn't matter
        BookTicket(destination: "Mumbai", passengerName: "Raj", seatType: "AC");
        Console.WriteLine("---");
        BookTicket(passengerName: "Sita", destination: "Pune");
    }
}

```

Q7. Method `ApplyDiscount` using ref parameter.

```
using System;
class Shop {
    static void ApplyDiscount(ref double price, double discountPercent) {
        double discount = (price * discountPercent) / 100;
        price -= discount;
    }
    static void Main() {
        double price = 1000.0;
        Console.WriteLine("Original Price: Rs." + price);
        ApplyDiscount(ref price, 20); // 20% discount
        Console.WriteLine("Price after 20% discount: Rs." + price);
    }
}
// Output:
// Original Price: Rs.1000
// Price after 20% discount: Rs.800
```

Q8. Method `DivideNumbers` using out parameters.

```
using System;
class MathOps {
    static void DivideNumbers(int a, int b, out int quotient, out int remainder) {
        quotient = a / b;
        remainder = a % b;
    }
    static void Main() {
        int q, r;
        DivideNumbers(17, 5, out q, out r);
        Console.WriteLine("17 / 5");
        Console.WriteLine("Quotient = " + q);
        Console.WriteLine("Remainder = " + r);
    }
}
// Output: Quotient = 3, Remainder = 2
```

Q9. Method `SUM` with default parameters.

```
using System;
class DefaultParams {
    static int SUM(int a, int b = 10, int c = 20) {
        return a + b + c;
    }
    static void Main() {
        Console.WriteLine("SUM(5) = " + SUM(5)); // 5+10+20
        Console.WriteLine("SUM(5,15) = " + SUM(5, 15)); // 5+15+20
        Console.WriteLine("SUM(5,15,25) = " + SUM(5, 15, 25)); // 5+15+25
    }
}
// Output: 35, 40, 45
```

Q10. Method `ADD` accepting dynamic data.

```
using System;
class DynamicAdd {
    static void ADD(dynamic a, dynamic b) {
```

```

var result = a + b;
Console.WriteLine($"ADD({a}, {b}) = {result} [Type: {result.GetType().Name}]");
}
static void Main() {
ADD(10, 20); // int
ADD(3.14f, 2.5f); // float
ADD("Hello, ", "World"); // string
ADD('A', 1); // char + int
}
}
// Output:
// ADD(10, 20) = 30 [Type: Int32]
// ADD(3.14, 2.5) = 5.64 [Type: Single]
// ADD(Hello, , World) = Hello, World [Type: String]
// ADD(A, 1) = 66 [Type: Int32]

```

Q11. Method `SquareNumber` with value parameter.

```

using System;
class Squares {
static int SquareNumber(int n) {
return n * n;
}
static void Main() {
Console.WriteLine("Square of 5 = " + SquareNumber(5));
Console.WriteLine("Square of 12 = " + SquareNumber(12));
Console.WriteLine("Square of 7 = " + SquareNumber(7));
}
}
// Output: 25, 144, 49

```

Q12. Method `AddMarks` using params array.

```

using System;
class Marks {
static int AddMarks(params int[] marks) {
int total = 0;
foreach (int m in marks) total += m;
return total;
}
static void Main() {
Console.WriteLine("Total (3 subjects): " + AddMarks(70, 80, 90));
Console.WriteLine("Total (5 subjects): " + AddMarks(65, 72, 88, 91, 55));
Console.WriteLine("Total (1 subject): " + AddMarks(100));
}
}
// Output: 240, 371, 100

```

Q13. Struct `Product` with user input.

```

using System;
struct Product {
public int Id;
public string Name;
public double Price;
public void Display() =>

```

```
Console.WriteLine($"ID: {Id} | Name: {Name} | Price: Rs.{Price}");
}
class Q13 {
static void Main() {
Product[] products = new Product[2];
for (int i = 0; i < 2; i++) {
Console.Write("Enter Product ID: ");
products[i].Id = int.Parse(Console.ReadLine());
Console.Write("Enter Name: ");
products[i].Name = Console.ReadLine();
Console.Write("Enter Price: ");
products[i].Price = double.Parse(Console.ReadLine());
}
Console.WriteLine("
--- Product Details ---");
foreach (var p in products) p.Display();
}
}
```

Q14. Single Inheritance: `Person` -> `Student`.

```
using System;
class Person {
public string Name;
public int Age;
public void ShowPerson() => Console.WriteLine($"Name: {Name}, Age: {Age}");
}
class Student : Person { // single inheritance
public int RollNo;
public double Marks;
public void ShowStudent() {
ShowPerson();
Console.WriteLine($"Roll No: {RollNo}, Marks: {Marks}");
}
}
class Q14 {
static void Main() {
Student s = new Student();
s.Name = "Asha"; s.Age = 20;
s.RollNo = 101; s.Marks = 88.5;
s.ShowStudent();
}
}
```

Q15. Multi-level Inheritance: `Student` -> `Exam` -> `Result`.

```
using System;
class Student2 {
public string Name;
public int RollNo;
}
class Exam : Student2 {
public int Marks1, Marks2, Marks3;
public int Total() => Marks1 + Marks2 + Marks3;
}
class Result : Exam {
public double Percentage() => (Total() / 300.0) * 100;
public string Grade() {
double p = Percentage();
if (p >= 75) return "Distinction";
if (p >= 60) return "First Class";
if (p >= 50) return "Second Class";
if (p >= 35) return "Pass";
return "Fail";
}
public void ShowResult() {
Console.WriteLine($"Name : {Name}");
Console.WriteLine($"Roll No : {RollNo}");
Console.WriteLine($"Total Marks: {Total()} / 300");
Console.WriteLine($"Percentage : {Percentage():F2}%");
Console.WriteLine($"Grade : {Grade()}");
}
}
class Q15 {
```

```

static void Main() {
Result r = new Result();
r.Name = "Rohan"; r.RollNo = 42;
r.Marks1 = 78; r.Marks2 = 85; r.Marks3 = 90;
r.ShowResult();
}
}

```

Q16. Hierarchical Inheritance: `Account` -> `SavingsAccount` & `CurrentAccount`.

```

using System;
class Account {
public string HolderName;
public double Balance;
public void ShowAccount() =>
Console.WriteLine($"Holder: {HolderName} | Balance: Rs.{Balance}");
}
class SavingsAccount : Account {
public double InterestRate = 4.0;
public void ShowSavings() {
ShowAccount();
Console.WriteLine($"Type: Savings | Interest Rate: {InterestRate}%");
}
}
class CurrentAccount : Account {
public double OverdraftLimit = 10000;
public void ShowCurrent() {
ShowAccount();
Console.WriteLine($"Type: Current | Overdraft Limit: Rs.{OverdraftLimit}");
}
}
class Q16 {
static void Main() {
SavingsAccount sa = new SavingsAccount();
sa.HolderName = "Priya"; sa.Balance = 50000;
sa.ShowSavings();
Console.WriteLine();
CurrentAccount ca = new CurrentAccount();
ca.HolderName = "Raj"; ca.Balance = 100000;
ca.ShowCurrent();
}
}

```

Q17. Interfaces `ISports` and `ICultural` implemented by `Student`.

```

using System;
interface ISports {
void ShowSportsAchievement();
}
interface ICultural {
void ShowCulturalAchievement();
}
class Student3 : ISports, ICultural {
public string Name;
public void ShowSportsAchievement() =>

```



```

Console.WriteLine($"{Name}: Won Gold Medal in 100m Sprint.");
public void ShowCulturalAchievement() =>
Console.WriteLine($"{Name}: Won First Prize in Classical Dance.");
}
class Q17 {
static void Main() {
Student3 s = new Student3 { Name = "Kavya" };
s.ShowSportsAchievement();
s.ShowCulturalAchievement();
}
}

```

Q18. Interface with default method.

```

using System;
interface IGreet {
void Greet(string name);
// Default interface method (C# 8.0+)
void DefaultGreet() {
Console.WriteLine("Hello! Welcome to our system.");
}
}
class EnglishGreet : IGreet {
public void Greet(string name) => Console.WriteLine($"Good Morning, {name}!");
}
class HindiGreet : IGreet {
public void Greet(string name) => Console.WriteLine($"Namaste, {name}!");
// Override default method
public void DefaultGreet() => Console.WriteLine("Swagat hai aapka!");
}
class Q18 {
static void Main() {
IGreet g1 = new EnglishGreet();
g1.Greet("Alice");
g1.DefaultGreet(); // uses interface default
Console.WriteLine();
IGreet g2 = new HindiGreet();
g2.Greet("Rahul");
g2.DefaultGreet(); // overridden
}
}

```

Q19. Class `Laptop` with read/write properties.

```
using System;
class Laptop {
    private string _brand;
    private double _price;
    public string Brand {
        get => _brand;
        set {
            if (!string.IsNullOrEmpty(value)) _brand = value;
            else Console.WriteLine("Brand cannot be empty.");
        }
    }
    public double Price {
        get => _price;
        set {
            if (value > 0) _price = value;
            else Console.WriteLine("Price must be positive.");
        }
    }
    public void Display() => Console.WriteLine($"Brand: {Brand}, Price: Rs.{Price}");
}
class Q19 {
    static void Main() {
        Laptop l = new Laptop();
        l.Brand = "Dell"; l.Price = 75000;
        l.Display();
        l.Price = 80000; // modify
        l.Display();
    }
}
```

Q20. Class `Order` with read-only OrderId property.

```
using System;
class Order {
    private static int _counter = 1000;
    // Read-only property - set only in constructor
    public int OrderId { get; private set; }
    public string Item;
    public Order(string item) {
        Item = item;
        OrderId = ++_counter; // auto-generated, cannot be changed outside
    }
    public void Display() => Console.WriteLine($"OrderId: {OrderId} | Item: {Item}");
}
class Q20 {
    static void Main() {
        Order o1 = new Order("Laptop");
        Order o2 = new Order("Phone");
        o1.Display();
        o2.Display();
        // o1.OrderId = 5000; // ERROR: cannot set - read only
    }
}
```

Q21. Class `PasswordManager` with write-only property.

```
using System;
class PasswordManager {
    private string _password;
    // Write-only property
    public string Password {
        set {
            if (value.Length >= 6)
                _password = value;
            else
                Console.WriteLine("Password too short (min 6 chars).");
        }
    }
    public bool Authenticate(string input) => _password == input;
}

class Q21 {
    static void Main() {
        PasswordManager pm = new PasswordManager();
        pm.Password = "secure123"; // set allowed
        // string p = pm.Password; // ERROR: get not accessible
        Console.WriteLine("Auth 'secure123': " + pm.Authenticate("secure123"));
        Console.WriteLine("Auth 'wrongpass': " + pm.Authenticate("wrongpass"));
    }
}
```

Q22. Class `Library` with indexer.

```
using System;
class Library {
    private string[] books;
    public Library(int size) { books = new string[size]; }
    // Indexer
    public string this[int index] {
        get {
            if (index >= 0 && index < books.Length) return books[index];
            return "Invalid index";
        }
        set {
            if (index >= 0 && index < books.Length) books[index] = value;
        }
    }
    public int Count => books.Length;
}

class Q22 {
    static void Main() {
        Library lib = new Library(3);
        lib[0] = "C# Programming";
        lib[1] = "Data Structures";
        lib[2] = "Operating Systems";
        for (int i = 0; i < lib.Count; i++)
            Console.WriteLine($"Book[{i}]: {lib[i]}");
    }
}
```

Q23. Class `Calculator` demonstrating method overloading.

```
using System;
class Calculator {
public int Add(int a, int b) => a + b;
public float Add(float a, float b) => a + b;
public double Add(double a, double b, double c) => a + b + c;
}
class Q23 {
static void Main() {
Calculator calc = new Calculator();
Console.WriteLine("Add(int): " + calc.Add(10, 20));
Console.WriteLine("Add(float): " + calc.Add(3.5f, 2.5f));
Console.WriteLine("Add(double): " + calc.Add(1.1, 2.2, 3.3));
}
}
// Output: 30, 6, 6.6
```

Q24. Unary minus (-) operator overloading for class `Temperature`.

```
using System;
class Temperature {
public double Celsius;
public Temperature(double c) { Celsius = c; }
// Overload unary minus
public static Temperature operator -(Temperature t) =>
new Temperature(-t.Celsius);
public void Display() => Console.WriteLine($"Temperature: {Celsius} C");
}
class Q24 {
static void Main() {
Temperature t1 = new Temperature(25);
Temperature t2 = -t1; // unary minus applied
t1.Display();
t2.Display();
}
}
// Output:
// Temperature: 25 C
// Temperature: -25 C
```

Q25. Binary minus (-) operator overloading for class `Point`.

```
using System;
class Point {
public int X, Y;
public Point(int x, int y) { X = x; Y = y; }
public static Point operator -(Point p1, Point p2) =>
new Point(p1.X - p2.X, p1.Y - p2.Y);
public void Display() => Console.WriteLine($"Point({X}, {Y})");
}
class Q25 {
static void Main() {
Point p1 = new Point(8, 6);
Point p2 = new Point(3, 2);
Point p3 = p1 - p2;
p1.Display();
p2.Display();
Console.WriteLine("p1 - p2 = "); p3.Display();
}
}
// Output: Point(5, 4)
```

Q26. Binary plus (+) operator overloading for `ComplexNumber`.

```
using System;
class ComplexNumber {
public double Real, Imaginary;
public ComplexNumber(double r, double i) { Real = r; Imaginary = i; }
public static ComplexNumber operator +(ComplexNumber c1, ComplexNumber c2) =>
new ComplexNumber(c1.Real + c2.Real, c1.Imaginary + c2.Imaginary);
public void Display() {
string sign = Imaginary >= 0 ? "+" : "-";
Console.WriteLine($"{Real} {sign} {Math.Abs(Imaginary)}i");
}
```

```

}
}
class Q26 {
static void Main() {
ComplexNumber c1 = new ComplexNumber(3, 4);
ComplexNumber c2 = new ComplexNumber(1, -2);
ComplexNumber c3 = c1 + c2;
Console.Write("C1 = "); c1.Display();
Console.Write("C2 = "); c2.Display();
Console.Write("C1 + C2 = "); c3.Display();
}
}
// Output: 4 + 2i

```

Q27. Overloading `>` and `<` operators for class `Box`.

```

using System;
class Box {
public double Length, Width, Height;
public Box(double l, double w, double h) {
Length = l; Width = w; Height = h;
}
public double Volume() => Length * Width * Height;
public static bool operator >(Box b1, Box b2) => b1.Volume() > b2.Volume();
public static bool operator <(Box b1, Box b2) => b1.Volume() < b2.Volume();
public void Display() => Console.WriteLine($"Box({Length}x{Width}x{Height}) Volume={Volume()}");
}
class Q27 {
static void Main() {
Box b1 = new Box(5, 4, 3);
Box b2 = new Box(3, 3, 3);
b1.Display(); b2.Display();
Console.WriteLine("b1 > b2 : " + (b1 > b2));
Console.WriteLine("b1 < b2 : " + (b1 < b2));
}
}
// b1 volume=60, b2 volume=27 -> b1 > b2 is True

```

Q28. Overloading ++ and -- operators for class `Counter`.

```

using System;
class Counter {
public int Value;
public Counter(int v) { Value = v; }
public static Counter operator ++(Counter c) => new Counter(c.Value + 1);
public static Counter operator --(Counter c) => new Counter(c.Value - 1);
public void Display() => Console.WriteLine("Counter = " + Value);
}
class Q28 {
static void Main() {
Counter c = new Counter(5);
c.Display();
c++; c.Display();
c++; c.Display();
c--; c.Display();
}
}

```

```
}  
}  
// Output: 5, 6, 7, 6
```

Q29. Base class `Shape` with overridden `Draw()` in `Circle` and `Rectangle`.

```
using System;
class Shape {
public virtual void Draw() => Console.WriteLine("Drawing a generic Shape.");
}
class Circle : Shape {
public double Radius;
public Circle(double r) { Radius = r; }
public override void Draw() =>
Console.WriteLine($"Drawing Circle with Radius = {Radius}");
}
class Rectangle : Shape {
public double Length, Width;
public Rectangle(double l, double w) { Length = l; Width = w; }
public override void Draw() =>
Console.WriteLine($"Drawing Rectangle {Length} x {Width}");
}
class Q29 {
static void Main() {
Shape[] shapes = { new Shape(), new Circle(7), new Rectangle(5, 3) };
foreach (Shape s in shapes)
s.Draw(); // polymorphic call
}
}
```

Q30. Abstract class `Appliance` with `TurnOn()` in `Fan` and `Light`.

```
using System;
abstract class Appliance {
public string Brand;
public Appliance(string brand) { Brand = brand; }
public abstract void TurnOn(); // must be implemented
public void ShowBrand() => Console.WriteLine($"Brand: {Brand}");
}
class Fan : Appliance {
public int Speed;
public Fan(string brand, int speed) : base(brand) { Speed = speed; }
public override void TurnOn() =>
Console.WriteLine($" {Brand} Fan turned ON at speed {Speed}.");
}
class Light : Appliance {
public string Color;
public Light(string brand, string color) : base(brand) { Color = color; }
public override void TurnOn() =>
Console.WriteLine($" {Brand} {Color} Light turned ON.");
}
class Q30 {
static void Main() {
Appliance[] appliances = { new Fan("Usha", 3), new Light("Philips", "White") };
foreach (Appliance a in appliances) {
a.ShowBrand();
a.TurnOn();
Console.WriteLine();
}
}
```



```
}  
}
```

Q31. Sealed class `SecuritySystem` with `Authenticate()`.

```
using System;  
  
sealed class SecuritySystem {  
    private string _accessCode = "SEC@2025";  
    public void Authenticate(string inputCode) {  
        if (inputCode == _accessCode)  
            Console.WriteLine("Access GRANTED. Welcome!");  
        else  
            Console.WriteLine("Access DENIED. Invalid code.");  
    }  
    public void ShowStatus() =>  
        Console.WriteLine("Security System is ACTIVE.");  
}  
  
// class AdvancedSecurity : SecuritySystem { } // ERROR: cannot inherit sealed class  
  
class Q31 {  
    static void Main() {  
        SecuritySystem sys = new SecuritySystem();  
        sys.ShowStatus();  
        sys.Authenticate("WRONG123");  
        sys.Authenticate("SEC@2025");  
    }  
}  
  
// Note: sealed keyword prevents inheritance - enhances security
```